END SEMESTER ASSESSMENT (ESA)
B.TECH. (CSE)
III SEMESTER

UE23CS251A – DIGITAL DESIGN & COMPUTER ORGANIZATION LABORATORY

PROJECT REPORT

ON

"Design and implement shift-left, shift-right & rotate operations for the 16-bit ALU using multiplexers."

Team No. 10

SUBMITTED BY

NAME	SRN
1) SUKRUTHA D	PES2UG23CS622
2) T G SANJANA	PES2UG23CS638
3) THANVI G ACHAR	PES2UG23CS652
4) SPOORTHI S H	PES2UG23CS593

AUGUST – DECEMBER 2024

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ELECTRONIC CITY CAMPUS,

BENGALURU – 560100, KARNATAKA, INDIA

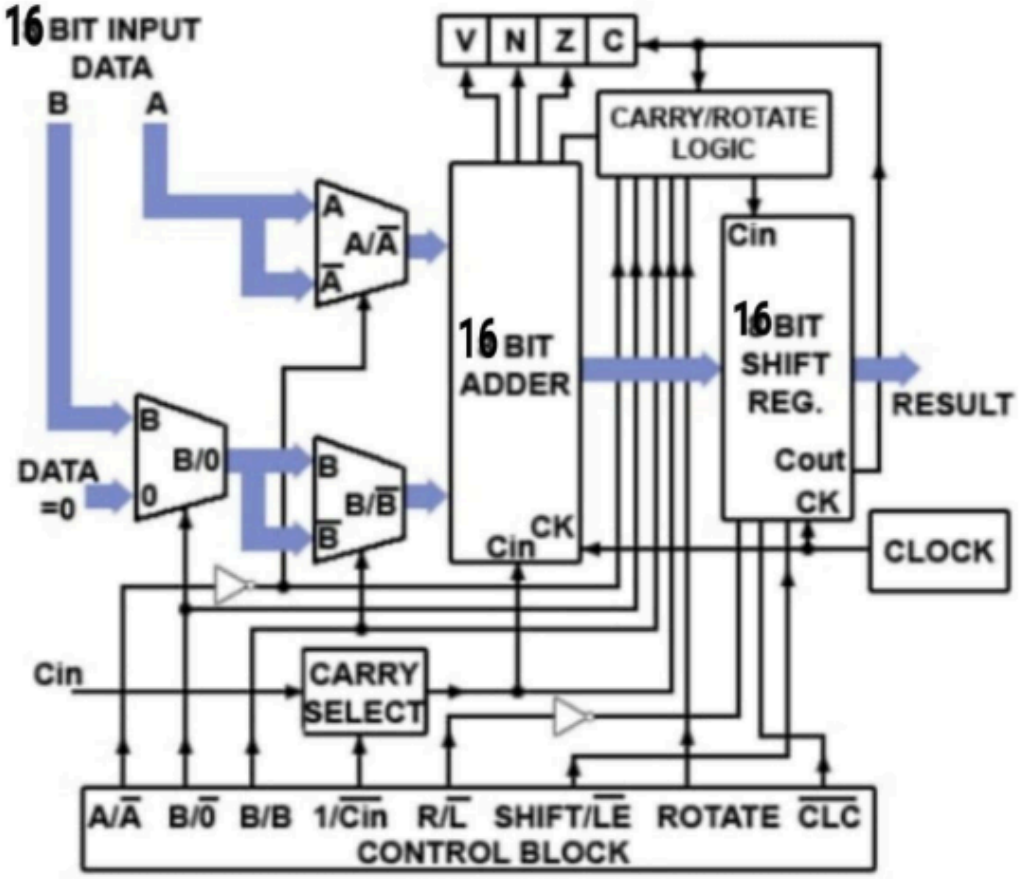
TABLE OF CONTENTS

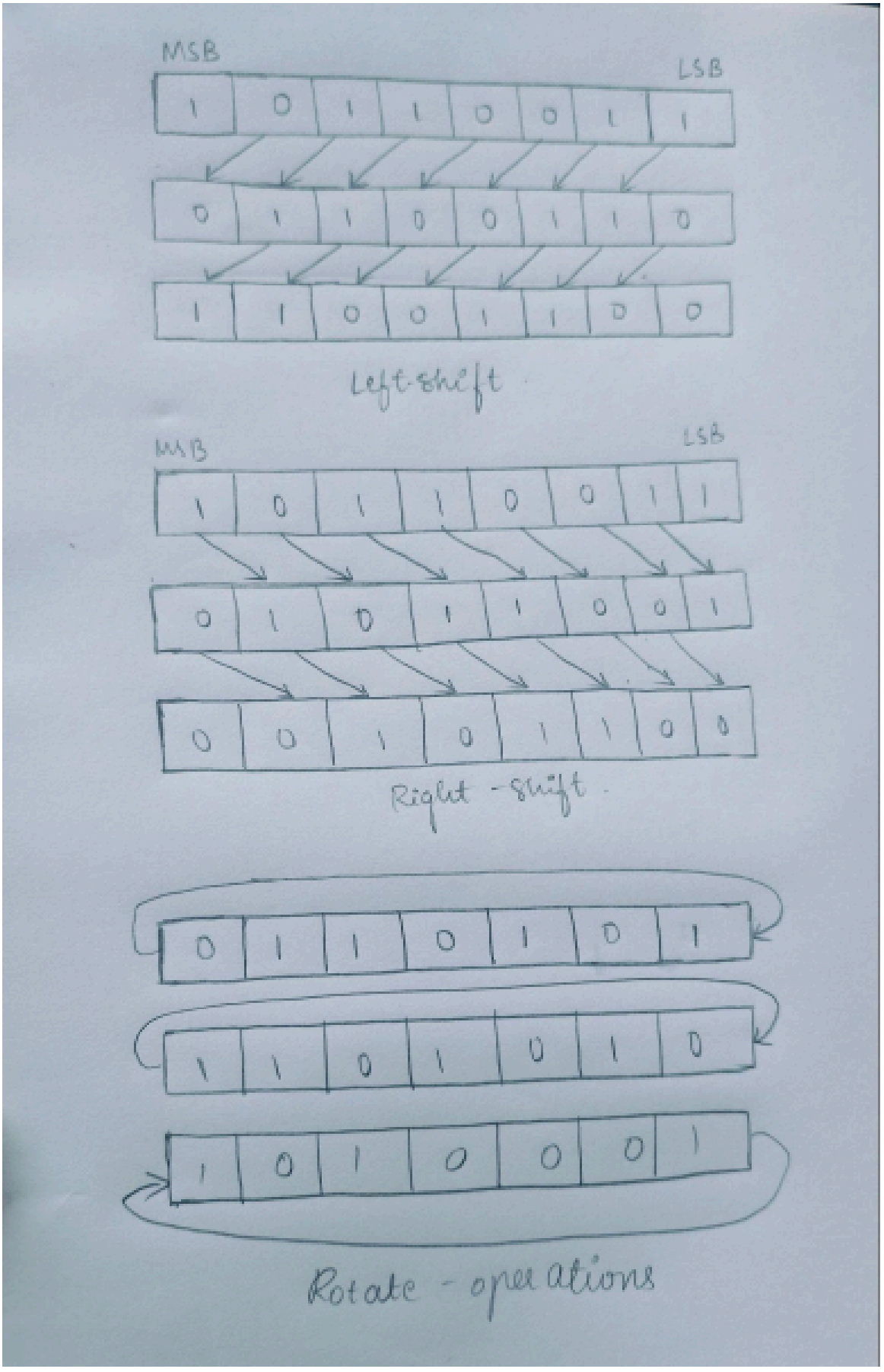
Sl.No	TOPIC	PAGE No
1.	ABSTRACT OF THE PROJECT	3
2.	CIRCUIT DIAGRAM	4
3.	MAIN VERILOG CODE	6
4.	TEST BENCH FILE	7
5.	SCREENSHOTS OF THE OUTPUT	9

ABSTRACT OF THE PROJECT:

This project focuses on designing and implementing shift-left, shift-right, and rotate operations in a 16-bit Arithmetic Logic Unit (ALU) using multiplexers. Multiplexers enable the selection between different shift and rotate operations based on control signals, ensuring a flexible and efficient design. The approach supports logical shifting and rotation in both directions while maintaining minimal hardware complexity. This work covers the design process, simulation, and performance evaluation of the proposed ALU operations, offering enhanced bitwise manipulation capabilities for digital systems

CIRCUIT DIAGRAM:




MSB
LSB
1 0 1 1 0 0 1 1
0 1 1 0 0 1 1 0
1 1 0 0 1 1 0 0
Left-shift
MSB
LSB
1 0 1 1 0 0 1 1
0 1 0 1 1 0 0 1
0 0 1 0 1 1 0 0
Right-shift
0 1 1 0 1 0 1
1 1 0 1 0 1 0
1 0 1 0 0 0 1
Rotate - operations

MAIN VERILOG CODE:

```
module ShiftRotateALU (
   input [15:0] A,         // 16-bit input
   input [3:0] Shift_Amount, // 4-bit shift amount
   input [1:0] S,          // 2-bit select signal
   output [15:0] Y         // 16-bit output
);
   wire [15:0] shl_result, shr_result, rol_result, ror_result;

   // Shift Left
   assign shl_result = A << Shift_Amount;

   // Shift Right
   assign shr_result = A >> Shift_Amount;

   // Rotate Left
   assign rol_result = (A << Shift_Amount) | (A >> (16 -
Shift_Amount));

   // Rotate Right
   assign ror_result = (A >> Shift_Amount) | (A << (16 -
Shift_Amount));

   // Select output based on control signal S
   assign Y = (S == 2'b00) ? shl_result :
              (S == 2'b01) ? shr_result :
              (S == 2'b10) ? rol_result :
              ror_result;
endmodule
```

TEST BENCH FILE:

```
module ShiftRotateALU_tb;
   // Inputs to the ALU
   reg [15:0] A;
   reg [3:0] Shift_Amount;
   reg [1:0] S;

   // Output from the ALU
   wire [15:0] Y;

   // Instantiate the ShiftRotateALU module
   ShiftRotateALU uut (
      .A(A),
      .Shift_Amount(Shift_Amount),
      .S(S),
      .Y(Y)
   );

   // Dump waveform data for GTKWave and print output
   initial begin
      // Set up VCD file for GTKWave
      $dumpfile("shift_rotate_alu.vcd");   // Specify the VCD
filename
      $dumpvars(0, ShiftRotateALU_tb);     // Dump all signals in
this module

      // Display header for terminal output
      $display("Time\t\tA\t\tShift_Amount\tS\t\tY");

      // Monitor statement to print each change of output
      $monitor("%0t\t%b\t%d\t\t%b\t\t%b", $time, A,
Shift_Amount, S, Y);
```

```
        // Test Case 1: Shift Left by 1
        A = 16'b0000_0000_0000_1010;  // Input data
        Shift_Amount = 4'd1;         // Shift by 1 position
        S = 2'b00;                   // Select shift left operation
        #10;                         // Wait for 10 time units

        // Test Case 2: Shift Right by 2
        A = 16'b0000_0000_0000_1010;  // Input data
        Shift_Amount = 4'd2;         // Shift by 2 positions
        S = 2'b01;                   // Select shift right operation
        #10;                         // Wait for 10 time units

        // Test Case 3: Rotate Left by 3
        A = 16'b1000_0000_0000_0001;  // Input data
        Shift_Amount = 4'd3;         // Rotate by 3 positions
        S = 2'b10;                   // Select rotate left operation
        #10;                         // Wait for 10 time units

        // Test Case 4: Rotate Right by 4
        A = 16'b0000_0000_0001_0000;  // Input data
        Shift_Amount = 4'd4;         // Rotate by 4 positions
        S = 2'b11;                   // Select rotate right operation
        #10;                         // Wait for 10 time units

        // Additional test cases can be added here

        $finish; // End simulation
    end
endmodule
```

SCREEN SHOT OF THE OUTPUT:

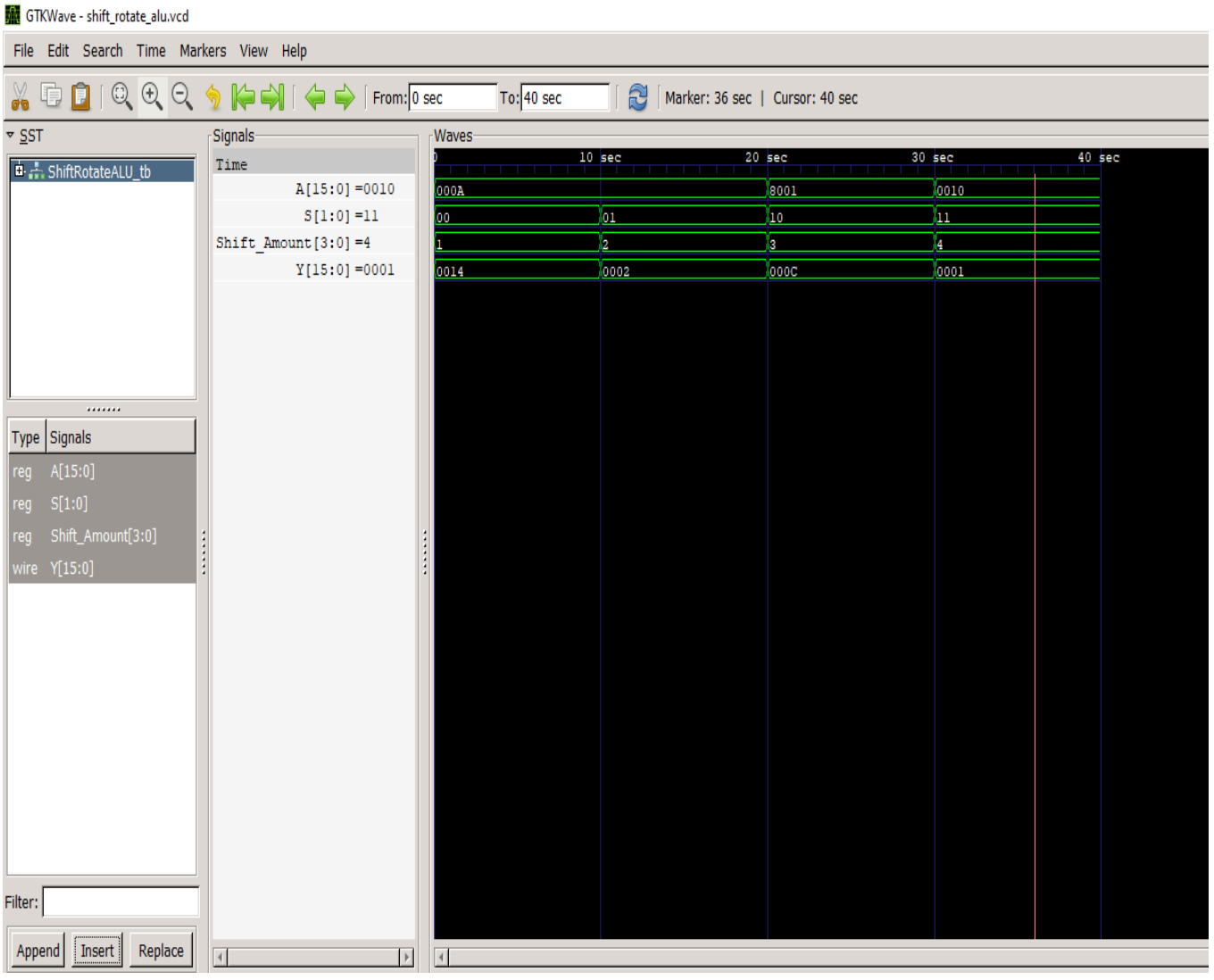
```
C:\iverilog\bin>iverilog -o test alu_mini.v alu_testbench1.v

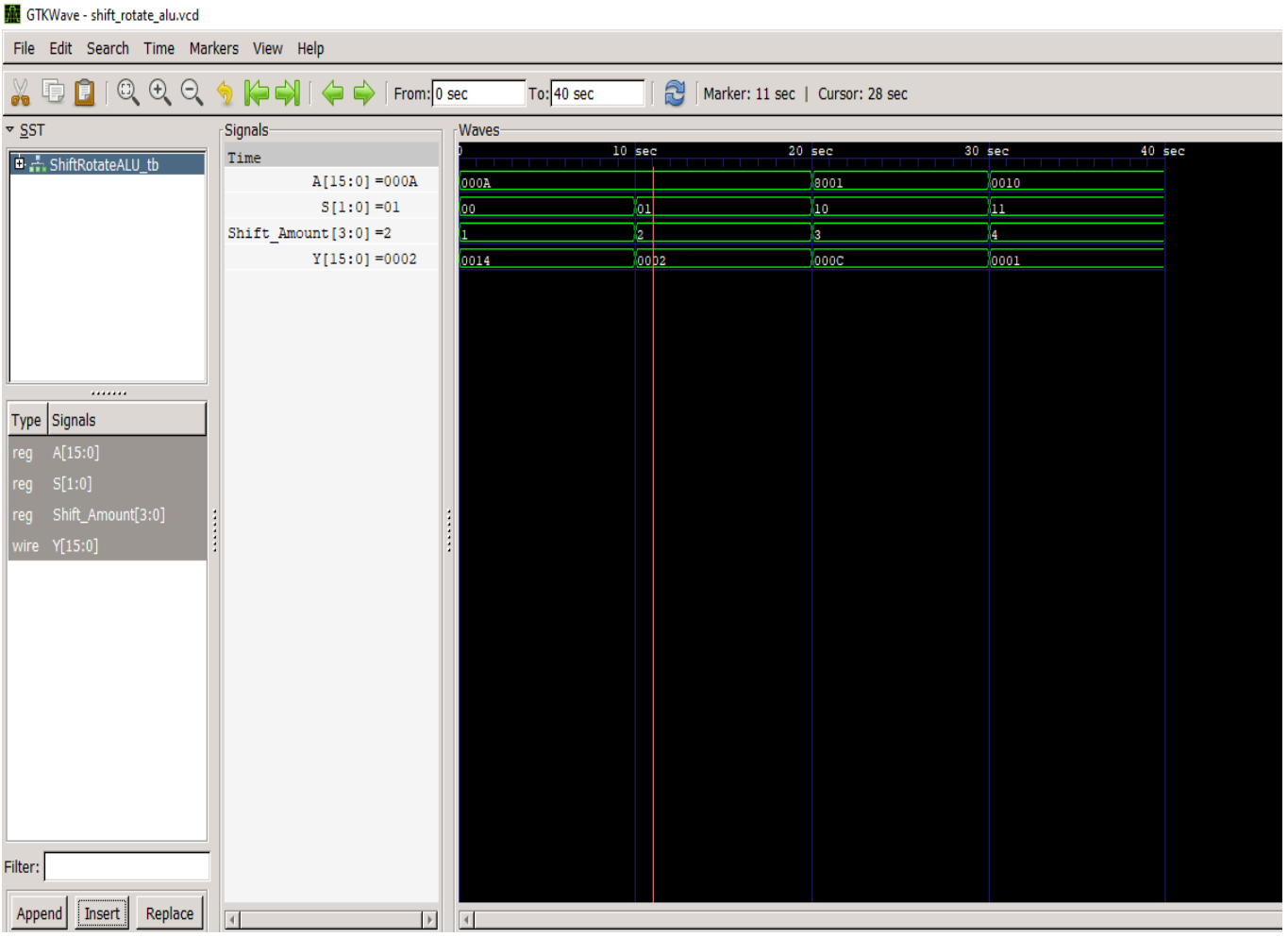
C:\iverilog\bin>vvp test
VCD info: dumpfile shift_rotate_alu.vcd opened for output.
Time            A               Shift_Amount    S               Y
0       0000000000001010         1              00              0000000000010100
10      0000000000001010         2              01              0000000000000010
20      1000000000000001         3              10              0000000000001100
30      0000000000010000         4              11              0000000000000001

C:\iverilog\bin>gtkwave shift_rotate_alu.vcd

GTKWave Analyzer v3.3.48 (w)1999-2013 BSI

[0] start time.
[40] end time.
```




GTKWave - shift_rotate_alu.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 40 sec
Marker: 11 sec | Cursor: 28 sec
SST
ShiftRotateALU_tb
Type Signals
reg A[15:0]
reg S[1:0]
reg Shift_Amount[3:0]
wire Y[15:0]
Filter:
Append Insert Replace
Signals
Time
A[15:0] =000A
S[1:0] =01
Shift_Amount[3:0] =2
Y[15:0] =0002
Waves
10 sec
20 sec
30 sec
40 sec
000A
8001
0010
00
01
10
11
1
2
3
4
0014
0002
000C
0001